

SIMATS ENGINEERING ASSESSMENT TOOLS 3

COURSE NAME: Theory of Computation

COURSE CODE: CSA1301

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks: 25

Duration: 1 Hour

CASE STUDY

Code of Conduct:

I Deepak V - 1924102 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Deepak

1. Consider a case study in parsing nested structured expressions using a Context-Free Grammar (CFG).
Given the grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S,$

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Designing a Grammar for Arithmetic Expressions

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as $a+b*c$.

Case Study Question:

Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

2. Mini Programming Language Parser Development

A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:

Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

3. Grammar Optimization for SQL Query Validation

A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:

Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

4. Natural Language Processing Chatbot

A chatbot system processes simple English sentences such as:

Subject + Verb + Object

Example: "Students learn TOC."

The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:

Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

5. Parser Design for HTML-like Markup Language

A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:

Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

Designing a Grammar for Arithmetic Expressions:

The initial grammar is ambiguous because expressions such as $a + b * c$ can have more than 1 parse tree,

Optimized CFG:

$E \rightarrow E + T \mid E - T \mid T$

$T \rightarrow T * F \mid T / F \mid F$

$F \rightarrow (E) \mid a \mid b \mid c$

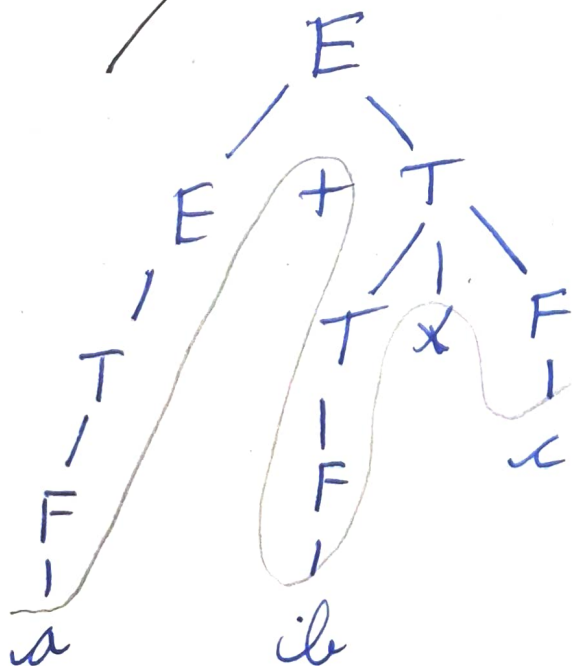
Here,

$E \rightarrow$ addition and subtraction

$T \rightarrow$ Multiplication and Division

$F \rightarrow$ operands and parenthesis

Ex. Now $a + b * c$ only has 1 parse tree,



$a + b * c$

2) Mini Programming Language Parser Development

The scripting language supports variable declarations; conditional statements, and loops

Optimized CFG:

Program $\rightarrow$ Statement List

Statement List $\rightarrow$ Statement Statement List / ϵ

Statement $\rightarrow$ Declaration
 | Assignment
 | If Statement
 | While Statement

Declaration $\rightarrow$ int id ;

Assignment $\rightarrow$ id = value ;

If Statement $\rightarrow$ if (condition) { Statement List }

While Statement $\rightarrow$ while (condition) { Statement }

Condition $\rightarrow$ id == value

 | id < value

 | id > value

value $\rightarrow$ id / number

Grammar Optimization for SQL Query Validation

A Simplified SQL grammar can be designed to validate commands containing SELECT, FROM, and an optional WHERE clause.

Optimized CFG:

Query $\rightarrow$ Select ColumnList FROM Table ~~WHERE~~

ColumnList $\rightarrow$ id

1 id, ColumnList

WhereClause $\rightarrow$ WHERE Condition

1 ϵ

Condition $\rightarrow$ id > number

1 id < number

1 id = number

Table $\rightarrow$ id.

The Grammar accepts queries such as:

$\rightarrow$ SELECT name FROM student

$\rightarrow$ SELECT name FROM student WHERE
mark > 50

Natural Language Processing Chatbot:

The chatbot processes simple English sentences using the structure:

Subject + Verb + object

CFC Design:

$S \rightarrow NP VP$

$NP \rightarrow N$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$N \rightarrow \text{Students} \mid \text{Students}'$

$V \rightarrow \text{learn} \mid \text{study}$

$NP \rightarrow TOC \mid \text{Computer Science}$

Therefore,

"Students learn TOC" is accepted
but;

"Learn Students TOC" is rejected.

Parser Design for HTML-like Markup Language:

A Markup language requires opening and closing tags to occur in the correct order. Recursive CFG productions can be used to support nested structures.

CFG for Nested Tags:

$S \rightarrow \text{Element}$

$\text{Element} \rightarrow \langle \text{tag} \rangle . \text{Content} \langle / \text{tag} \rangle$

$\text{Content} \rightarrow \text{Element Content}$

$\quad \quad \quad | \text{text} . \text{Content}$

$\quad \quad \quad | \epsilon$

$\text{text} \rightarrow \text{text} .$

Important Production:

$\Rightarrow \text{Element} \rightarrow \langle \text{tag} \rangle \text{Content} \langle / \text{tag} \rangle$

Example:

$\langle \text{tag} \rangle$

$\langle \text{tag} \rangle \text{text} \langle / \text{tag} \rangle$

$\langle / \text{tag} \rangle$

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandabl e with minor issues	Somewhat unclear or poorly structured	Difficult to understand

86/100